SIC 8 · BIG DATA ENGINEERING · GRADUATION PROJECT

Smart Grid Monitor

Technical documentation of a big-data pipeline that turns 100,000 smart electricity meters into alerts for electricity theft, overloaded transformers and voltage drops

Project 04 · Smart Energy, Power Grids & Utilities

High-Scale Smart Meter (AMI) Ingestion & Cloud/Local Data Platform

TEAM

Abdelrahman Adel · Data generation, Kafka streaming and the Bronze layer

Zyad Abdalla · Spark cleaning and the Silver layer

Mohamed Samir · Snowflake warehouse, dbt models and the threat checks

Abdulrhman Mohamed · AWS, integration, Airflow, Power BI and the end-to-end run

October 2026

Contents

1. The project in one page
2. Electricity in five minutes
3. The problem and our goals
4. The solution at a glance
5. The team and who did what
6. The environment: where everything runs and why
7. The tools we picked and why
8. The data we simulate
9. The pipeline, phase by phase
10. The data layers: Bronze, Silver and Gold
11. The data model
12. The threats we catch and how we recognise them
13. Orchestration with Airflow
14. Data quality and testing
15. The Power BI dashboard
16. Results and measured numbers
17. Key decisions and why we made them
18. Problems we hit and how we fixed them
19. Limitations and what we would do next
20. How to run the project
21. How we meet requirements

1. The project in one page

Every home in a city has an electricity meter. A **smart meter** sends its reading automatically every 15 minutes, so a power company with 100,000 homes receives **9.6 million readings a day**. Hidden in that flood of numbers are three expensive problems: people stealing electricity, street transformers being pushed past their limit, and power lines whose voltage drops too low.

We built a complete data platform that receives these readings as a live stream, cleans them, stores them safely in a cloud data lake, models them in a cloud data warehouse, runs three detection rules, and shows the results on a dashboard. A scheduler runs the batch part of the pipeline by itself every 15 minutes.

Because real meter data is private, we wrote our own simulator. It creates a realistic city grid in Cairo and Giza and secretly plants 25 problems in the data. It writes the list of planted problems to a separate **answer key** that the pipeline never sees. At the end we compare what the pipeline found with the answer key, so our accuracy is measured, not assumed.

Results at a glance

What	Result
Smart meters simulated	100,000 meters on 500 transformers, in 5 districts of Cairo and Giza
Reading interval	Every 15 minutes (96 readings per meter per day)
Readings streamed through Kafka	9,649,221 meter messages + 48,000 transformer messages, 0 failed
Clean readings in the warehouse	9,591,495 meter readings after removing duplicates and broken messages
Threats planted vs found	25 planted, 25 found, 0 false alarms, 0 missed (precision and recall = 100%)
Automation	Airflow runs clean → load → model → test every 15 minutes; 5 required batch runs proven
Data quality tests	15 automated tests, all passing; 1 extra test fails on purpose to demo the quality gate
Dashboard	Power BI, 4 pages: Overview, Alert Map, Alert Log, Accuracy

Table 1: Headline results of the final run

2. Electricity in five minutes

You do not need to be an electrical engineer to follow this project. Four ideas are enough, and they all work like water in pipes.

Word	Like water…	What it means here	Unit
Voltage	The water pressure in the pipe	How strongly electricity is pushed to the homes. Normal is about 225–230 volts. Too low and fridges and air conditioners struggle	V (volts)
Power	How much water flows right now	How much electricity is being used at this moment. Goes up when the AC turns on	kW (kilowatts)
Energy	The total water used over a period	Power used over time. This is what the electricity bill counts. kW × hours = kWh	kWh (kilowatt-hours)
Reactive power	Water that sloshes back and forth without doing work	Extra power that motors (ACs, fridges) pull but don't really use. The handbook asks us to record it	kvar
Capacity	How big the pipe is	The most power a transformer can deliver safely. Above it, it overheats and may trip	kW

Table 2: The only electricity words you need

How electricity reaches a home

Electricity travels from big stations down to homes through a chain. Our simulator builds this exact chain, so every reading can be traced to a place on the map.



Figure 1: The grid chain we simulate, from the big station down to the home

The **transformer** is the most important box in this project. It is the grey box on the street that feeds about 200 homes. It measures everything that leaves it, and each home's meter measures what that home uses. Comparing the two is how we catch theft (section 12).

What a smart meter (AMI) is

AMI stands for Advanced Metering Infrastructure: smart meters plus the network that collects their readings. Instead of an employee reading the meter once a month, the meter reports by itself every 15 minutes. Each reading carries power (kW), reactive power (kvar), voltage (V) and the energy used in those 15 minutes (kWh).

3. The problem and our goals

The business problem

A power company needs two views of the same data at the same time. It needs the **big picture** (how much electricity the whole city uses each day, and when the peak is) and it needs to **zoom in** on local trouble (one transformer that is about to burn out, or one street where energy disappears). With millions of readings a day, nobody can do this by hand.

Who uses the result

- **Grid operations team:** needs to know which transformers are overloaded or have low voltage, so they can send a crew before an outage.
- **Loss-control (anti-theft) team:** needs a short list of areas where energy goes missing, so inspectors visit the right streets.
- **Management:** needs daily demand and peak numbers to plan capacity.

The three threats

Threat	What happens in real life	Why it costs money	How we see it in the data
Electricity theft	Someone bypasses or tampers with their meter, so it records less than they really use	The company pays for energy it never gets paid for	The transformer sent out clearly more energy than its homes' meters recorded
Transformer overload	Too many homes run heavy loads (ACs on a hot evening) at the same time	The transformer overheats, ages fast, and can trip, cutting power to the whole street	The transformer's load stays near or above its capacity
Line drop	A damaged, loose or weak cable makes the voltage fall	Appliances fail or get damaged; a crew must find and fix the cable	Voltage stays below the safe minimum for an hour

Table 3: The three threats in plain words

Measurable goals

Goal	How we measure it	Reached?
Simulate at least 100,000 meters reporting every 15 minutes	Distinct meter IDs in the data; every timestamp on :00, :15, :30 or :45	Yes: 100,000 meters, 96 slots per day
Record active power, reactive power and voltage	Columns kw, kvar, voltage in every reading	Yes
Plant all three threat types	Rows in the answer key (sim_truth)	Yes: 10 theft, 10 overload, 5 line-drop transformers
Show transformer peak loading, anomaly flags and daily grid demand	Visible on the dashboard and queryable in Snowflake	Yes
Detect the planted threats without seeing the answer key	Precision and recall against sim_truth	Yes: 100% / 100%
Automate raw-to-dashboard and make it reproducible	Airflow runs; same config gives the same data	Yes

Table 4: Our goals and how we checked them

4. The solution at a glance

The pipeline has eight stops. The left side runs on a laptop in Docker containers; the data lake (AWS S3) and the warehouse (Snowflake) are in the cloud; the dashboard runs in Power BI Desktop on Windows. Each box shows who built it.

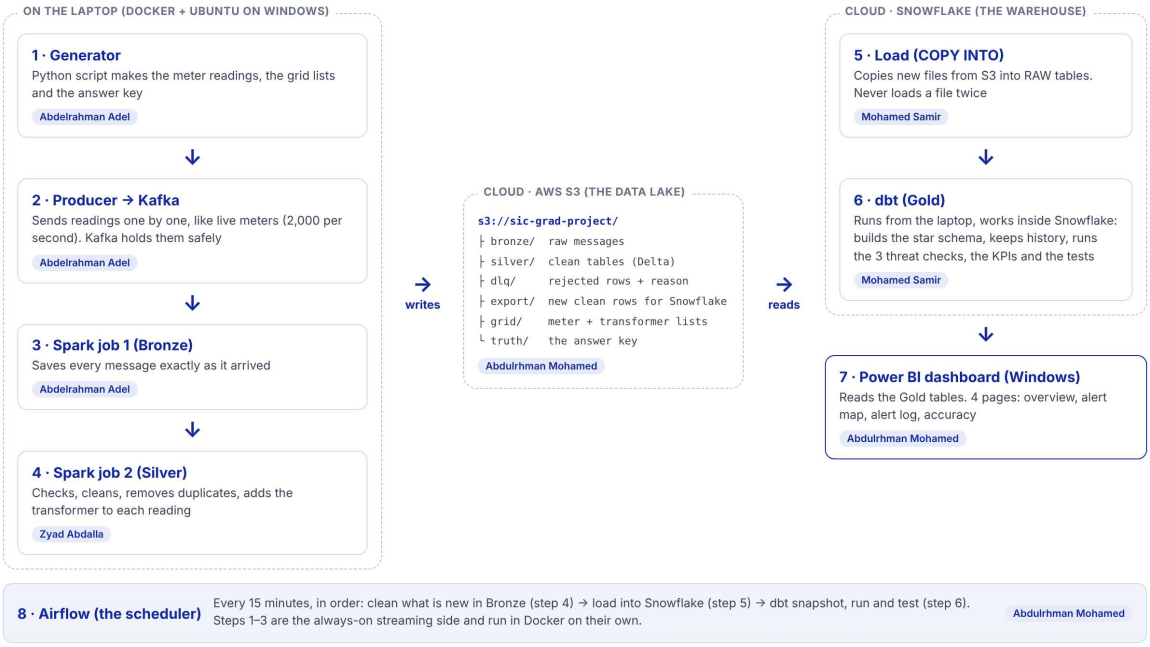


Figure 2: The end-to-end architecture, with the owner of each part

The whole story in one paragraph

The **generator** creates a day of meter readings plus the grid lists and the answer key. The **producer** replays the readings into **Kafka** at high speed, as if 100,000 meters were reporting live. **Spark job 1** saves every Kafka message untouched into the **Bronze** layer on S3. **Spark job 2** reads Bronze, rejects broken messages into a dead-letter folder, removes duplicates, attaches each meter to its transformer, and writes clean **Silver** tables plus an **export** copy for the warehouse. **Snowflake** loads the export files into raw tables. **dbt** builds the **Gold** layer: a star schema, the history of meter changes, the three threat checks, the KPI tables and the accuracy score. **Power BI** shows the result. **Airflow** runs the batch steps (clean → load → model → test) every 15 minutes.

Stream and batch, together

Part	Stream or batch	Why
Meter and transformer readings → Kafka → Bronze	Stream (nonstop)	Readings arrive every 15 minutes from every meter and must never wait
Bronze → Silver cleaning	Stream (micro-batch every minute) or batch (Airflow, every 15 min)	Same code, two modes: continuous for the live demo, on schedule for normal running
Grid lists (who is connected to whom)	Batch, once a day	The grid rarely changes, so one fresh copy a day is enough
Snowflake load + dbt models + tests	Batch, every 15 minutes (Airflow)	Each run processes only what is new since the last run

Table 5: Where we stream and where we batch

5. The team and who did what

We split the pipeline into four parts, one per person. Each person owns their part from start to end, so each can explain it in detail. We worked in order (data → cleaning → warehouse → serving), and while waiting for the person before them, each member built and tested their part on small sample files in the agreed formats.

Member	Role	What they built	Main evidence
Abdelrahman Adel	Data generation and streaming	The simulator (generator.py + config.yaml) with the planted threats and the answer key; Kafka in Docker with two topics; the producer that replays readings at a set speed; Spark job 1 that writes Bronze; the run script (run_m1.sh) with smoke, full-day and demo modes; uploads of the grid files and the answer key	Producer logs (0 failed messages), Bronze files on S3, smoke test pass
Zyad Abdalla	Distributed processing (Silver)	Spark job 2: schema and value checks, dead-letter queue with reasons, de-duplication, enrichment with the grid map, Delta Lake MERGE into Silver, export files partitioned by date and hour, and a run-once mode for Airflow	Silver batch logs, DLQ folders, no duplicate keys in Silver
Mohamed Samir	Warehouse and detection	Snowflake setup (stages, file formats, storage integration with AWS); COPY INTO loading; the dbt project: SCD2 snapshot, star schema, the three threat checks, KPI tables, accuracy models and data tests	dbt run (14 models) and dbt test (15 tests) passing; accuracy table
Abdulrhman Mohamed	Cloud, integration, orchestration and serving	AWS account, bucket, IAM users and policies, and the Snowflake access role; joined the four parts and fixed the mismatches between them; ran the full pipeline end to end on his own accounts; Airflow DAG and the five batch runs; Power BI model, measures and dashboard; demo video; documentation	Airflow runs and logs, Snowflake counts, the dashboard, this document

Table 6: Team contribution summary

6. The environment: where everything runs and why

We chose a **hybrid**: the heavy processing runs on a laptop for free, and storage and the warehouse run in the cloud where they are cheap, shared and reliable.

Component	Runs on	Version	Why there
Generator, producer	Laptop, Docker container	Python 3	Needs no cloud; runs in seconds
Kafka (message broker)	Laptop, Docker container	apache/kafka 3.9.1 (KRaft mode, no ZooKeeper)	One command to start; the handbook's first choice for live data
Spark jobs 1 and 2	Laptop, Docker containers	Spark 3.5.5, Java 17, Delta Lake 3.2.0	Free, fast to debug; the same code could move to AWS EMR or Glue unchanged
Data lake	AWS S3, bucket sic-grad-project	Region eu-north-1 (Stockholm)	Durable, cheap, and Snowflake can read it directly
Data warehouse	Snowflake (cloud)	Region AWS eu-central-2 (Zurich), X-Small warehouse, auto-suspend after 60 s	Fast SQL, built-in loading from S3, time travel; we had trial credits
dbt (data models)	Laptop (Ubuntu), runs SQL inside Snowflake	dbt with the Snowflake adapter	dbt only sends SQL; the work happens in Snowflake
Airflow (scheduler)	Laptop (Ubuntu), own Python environment	Apache Airflow 2.10.5, Python 3.12	Needs access to Docker and dbt on the same machine
Power BI	Windows laptop	Power BI Desktop, Import mode	Free desktop tool with a built-in Snowflake connector

Table 7: Where each part runs

Why a laptop for processing, not the cloud

1. **Cost and credits.** Building means dozens of test runs a week. On a laptop they are free; on AWS Glue or EMR each run costs money and starts slowly.
2. **Faster debugging.** Change one line, run it, see the error in seconds.
3. **Safer access.** Spark only needs a key for one bucket, not access to the AWS console.
4. **Same code.** It is standard PySpark, so it can move to a cloud cluster later without changes.

The handbook states that local implementation carries no disadvantage just because it avoids paid cloud services, and that infrastructure cost does not earn marks.

Ubuntu on Windows (WSL) and Docker

The laptop runs Windows. The scripts are Linux shell scripts, so we run them inside **Ubuntu on Windows (WSL 2)**. Kafka and Spark run in **Docker** containers, so they start with one command and behave the same on any machine. Power BI runs on the Windows side and connects to Snowflake over the internet.

Security and access

Who / what	Access	Why
Account owner (Abdulrhman Mohamed)	Full AWS console	Owns the bucket, the keys and the Snowflake role
Each pipeline user (data, Spark, pipeline)	An access key that works on one bucket only, no console login	If a key leaks, only that key is deleted; nobody can create servers or spend credits
Snowflake	No key at all. A read-only AWS role limited to export/, grid/ and truth/	Snowflake can read what it needs and nothing else
dbt and Airflow	Snowflake user DBT_USER with role DBT_ROLE (reads RAW, builds CORE and MARTS)	Least privilege: it cannot touch anything else
Power BI	Snowflake user PBI_USER with role PBI_ROLE (reads MARTS only)	The dashboard can only read the final tables

Table 8: Who can access what

Keys live only in a local .env file that is excluded from Git, and are shared privately, never in group chats.

7. The tools we picked and why

Every tool has exactly one job. For each one we also say what we considered instead and why we did not use it.

Tool	Its job, in simple words	Why we picked it	Alternative we did not use
Python	Makes the simulated meter data	The handbook requires a custom generator	A downloaded CSV does not meet the requirement
Docker	Starts each tool in a ready-made box with one command	Same setup on every laptop; no 'works on my machine' problems	Installing Kafka and Spark by hand
Kafka	A waiting room where readings wait safely until Spark picks them up	Built for high-speed live events; never loses a message; handbook's first choice	NiFi is better suited to moving files than live events
PySpark	The worker that checks, cleans and reshapes big data	Handles both streaming and batch with the same code; handbook requirement	Plain Python would not scale to millions of rows
AWS S3	The online hard drive for the data lake	Cheap, durable, read directly by Snowflake	MinIO keeps everything local with no cloud copy
Delta Lake	Turns folders of files into real tables	ACID writes, MERGE (upsert), time travel; no extra service needed	Iceberg needs a separate catalog service
Snowflake	The cloud warehouse: stores tables and runs SQL	Loads from S3 with one command, scales on demand, trial credits available	Databricks Free Edition could not read our own S3 bucket
COPY INTO	Snowflake's built-in loader	Free, simple, remembers loaded files so nothing loads twice	Airbyte needs about 8 GB of RAM on a laptop
dbt	A recipe book of SQL models that runs them in the right order and tests them	Gives history (SCD2), tests and a lineage diagram almost for free	Hand-written SQL scripts, with more manual work and no tests
Airflow	The alarm clock that runs each step on time and in order	Dependencies, retries and logs; handbook requirement	cron has no order, retries or history
Power BI	Draws the dashboard	Free desktop version, Snowflake connector, map visual, and the team knows it	Superset needs its own server

Table 9: Tool choices and the alternatives we rejected

8. The data we simulate

Real smart-meter data is private and has no record of which homes were really stealing. So we wrote our own simulator, generator.py, controlled by one settings file, config.yaml. It builds a realistic grid, makes normal usage, plants the threats on purpose, adds messy data on purpose, and writes the answer key.

8.1 The simulated world

Thing	How many	ID looks like
Governorates	2 (Cairo, Giza)	Cairo
Districts	5: Dokki, Nasr City, Maadi, Heliopolis, Shubra	Dokki
Substations	5 (one per district)	S-01
Feeders (big cables)	25 (5 per substation)	F-01
Transformers	500 (20 per feeder)	T-0117
Home and shop meters	100,000 (about 200 per transformer; 8% are shops)	M-048213
Simulated days	2: 1 Aug 2026 (fully streamed) and 2 Aug 2026 (grid changes + demo slots)	

Table 10: The size of the simulated grid

8.2 The datasets and their columns

Dataset 1: meter readings (live stream, every 15 minutes)

{"meter_id":"M-048213","ts":"2026-08-01T19:15:00Z","kw":2.84,"kvar":0.61,"voltage":221.4,"kwh":0.71}

Column	Meaning	Normal values
meter_id	Which home or shop	M-000001 to M-100000
ts	Time of the reading (UTC)	Always on :00, :15, :30 or :45
kw	Power being used right now	0.2–6 kW for homes, 2–20 kW for shops
kvar	Reactive power (wasted by motors)	About 0.3–0.5 × kw
voltage	Voltage at the home	About 209–231 V
kwh	Energy used in these 15 minutes	kw × 0.25 hours

Dataset 2: transformer readings (live stream, every 15 minutes)

{"transformer_id":"T-0117","ts":"2026-08-01T19:15:00Z","kw":412.0,"kwh":103.0,"voltage":224.9}

Column	Meaning
transformer_id	Which transformer
kw	Its real load: everything its homes really use + 6% normal cable loss + any stolen power
kwh	Energy it really sent out in these 15 minutes
voltage	About 228 V, minus a small drop that grows with load

Dataset 3: meter list (batch file, one per day)

```
meter_id,transformer_id,tariff,customer_name,address,lat,lon
M-048213,T-0117,residential,Ahmed Mahmoud,12 Tahrir St Dokki,30.0391,31.2118
```

Says which transformer feeds each meter, whether it is a home or a shop, and where it is. Names and addresses are invented. Each day, a few things change on purpose so we can show history tracking (SCD): about 0.1% of meters move to another transformer, about 0.5% change tariff, and a few name typos are fixed.

Dataset 4: transformer list (batch file, one per day)

```
transformer_id,feeder_id,substation_id,district,governorate,capacity_kw,lat,lon
T-0001,F-01,S-01,Dokki,Giza,830,30.044975,31.222315
```

Gives each transformer its place in the chain, its district and its **capacity**. Capacity is sized so that a normal evening peak uses about 75% of it.

Dataset 5: the answer key (sim_truth)

```
{"anomaly_id":"A-0031","event_type":"overload","entity_id":"T-0003","transformer_id":"T-0003",
 "start_ts":"2026-08-01T18:00:00Z","end_ts":"2026-08-01T21:00:00Z","factor":1.45}
```

One line per planted problem (815 lines: 300 thieving meters on 10 transformers, 10 overloads, 5 line drops and 500 silent meters). It is uploaded to its own folder and used **only** by the final accuracy model. The detection rules never read it, so they can't cheat.

8.3 How normal usage behaves

- Homes: low at night, a small morning bump, a big evening peak from about 19:00 to 23:00 (air conditioners).
- Shops: busy from about 10:00 to 22:00.
- Random noise on every reading, so no two homes look the same.
- A normal transformer peaks at about 70–80% of its capacity, so overload only appears where we plant it.

8.4 What we plant on purpose

Planted	What the simulator does	Where and when
Overload	The homes on a transformer use 45% more power (ACs on). Its load climbs to about 110% of capacity	10 transformers, 18:00–21:00 on day 1
Line drop	Voltage is 12% lower, falling to about 197–200 V	5 transformers, 06:00–10:00 on day 1
Theft	30 homes on a transformer get meters that record only 20–40% of their real use. The transformer still sees all of it	10 transformers (300 meters), all day
Silent meters	A meter stops sending for 1–3 hours	500 meters
Duplicates	The same reading sent twice	0.5% of messages
Broken messages	Cut off mid-message, empty meter ID, negative kW, text instead of a number, or no time	0.1% of messages
Late readings	The reading keeps its real time but arrives 15 minutes to 2 hours late	1% of readings

Table 11: Threats and data problems planted by the simulator

Why the overload raises the **homes' usage** and not just the transformer: in real life it is the homes that use more. If only the transformer number went up, the homes' total would stay flat and it would look like 30% of the energy was missing, a fake theft.

8.5 Reproducible by design

Everything comes from config.yaml: the number of meters and transformers, the days, the normal load, the cable loss, every planted threat and every kind of messy data. It uses a fixed random seed (42), so the same settings always produce exactly the same data. A small development mode (1,000 meters on 5 transformers) was used while building; the final run used the full 100,000 meters.

9. The pipeline, phase by phase

This section walks through the pipeline in the order the data travels. For each phase: who built it, what goes in, what comes out, what happens inside, why we did it that way, and what we got.

Phase 1 · Generate the data

Owner	Abdelrahman Adel
Tools	Python, NumPy, PyYAML, Docker
Takes in	config.yaml
Produces	One file of meter readings and one of transformer readings per day (JSON lines), daily grid lists (CSV), the answer key (JSON lines), and a run summary (run_metrics.json)

The run script generates the data, uploads the grid lists to S3 grid/ and the answer key to truth/, then starts streaming. A **smoke test** mode sends a small sample through the whole path first and checks the counts (100 meter and 5 transformer messages in, the same out), so we know everything is connected before the long run.

Result: the full day 1 data (about 9.65 million meter messages including duplicates and broken ones, and 48,000 transformer readings), grid files for both days, and 815 answer-key lines.

Phase 2 · Ingest: producer → Kafka

Owner	Abdelrahman Adel
Tools	Kafka 3.9.1 in Docker (KRaft mode), Python Kafka producer
Takes in	The daily reading files
Produces	Two Kafka topics: meter.readings and transformer.readings

The **producer** reads the files in time order and sends each reading as one Kafka message, as if 100,000 meters were reporting live. It sends meter and transformer readings in parallel (two threads) and limits the speed to **2,000 meter readings per second** (10 transformer readings per second) so the laptop keeps up.

- **Safe sending:** every message waits for Kafka to confirm it (acks = all), with up to 5 automatic retries.
- **Logs:** the producer prints how many messages were queued, confirmed and failed for each file.
- **Why Kafka:** it holds every message until Spark reads it. If Spark is slow or restarts, nothing is lost. This saved us once (Phase 3).

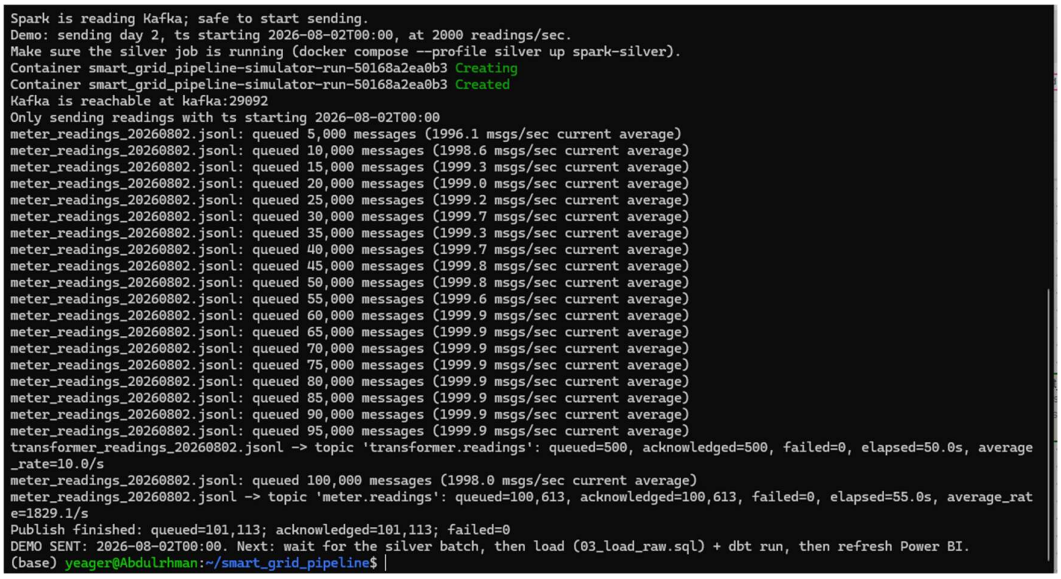


Figure 3: Producer output for one 15-minute slot of day 2: 101,113 messages sent, 0 failed

Result: 9,649,221 meter messages and 48,000 transformer messages delivered, 0 failed, in about 80 minutes.

Phase 3 · Bronze: keep the raw data

Owner	Abdelrahman Adel
Tools	Spark Structured Streaming in Docker
Takes in	The two Kafka topics
Produces	S3 bronze/meter_readings/ and bronze/transformer_readings/ (Parquet)

Spark job 1 reads Kafka every 60 seconds and writes every message **exactly as it arrived**, together with where it came from (topic, partition, offset) and when Kafka received it. Nothing is parsed, fixed or dropped. A checkpoint folder remembers how far it has read, so a restart continues where it stopped.

> **In simple words:** Bronze is the black-box recorder. Whatever happens later, we can always go back to exactly what arrived.

Is Bronze still raw if it is Parquet? Yes. 'Raw' is about the content, not the file type. The original JSON text sits untouched inside one column (raw_value). Parquet is only a smaller, safer envelope.

What went wrong and how we fixed it: on the first full run, Spark was told to read only *new* Kafka messages and it started about 20 seconds after the producer, so the first 39,367 meter and 197 transformer messages were skipped. Because Kafka still held them, we re-read them into Bronze and lost nothing. We then changed the run script to wait until Spark is ready before sending.

Phase 4 · Silver: check, clean and enrich

Owner	Zyad Abdalla
Tools	PySpark Structured Streaming, Delta Lake 3.2.0
Takes in	S3 bronze/ + the meter list in grid/
Produces	S3 silver/ (Delta tables), dlq/ (rejected rows + reason), export/ (new clean rows for Snowflake)

Spark job 2 reads new Bronze files in small batches. It runs in two modes with the same code: **continuously** (a batch every minute, used for the live demo) or **run once and stop** (used by Airflow every 15 minutes). Each batch does six things, in order:

1. **Read the JSON** with a fixed schema (enforced columns and types). Text is converted to numbers safely: a value that can't be converted becomes empty instead of crashing the job.
2. **Check every reading.** Anything that fails a rule goes to the dead-letter queue (dlq/) with the reason, the original text and its Kafka position, so it stays visible and can be fixed and replayed later.
3. **Remove duplicates** using the business key (meter + time, or transformer + time). Inside a batch it keeps the first copy; across batches the Delta MERGE makes sure a key is stored only once.
4. **Add the transformer to each meter reading** by joining with that day's meter list. The list is small, so Spark sends a copy to every worker (a broadcast join) instead of shuffling the big data.
5. **Merge into Silver** (Delta Lake MERGE: update if the key exists, insert if not), partitioned by date.
6. **Write only the brand-new rows to `export/`** as Parquet, partitioned by date and hour, for Snowflake to load.

Reject reason	Example of what is caught
malformed_json	A message cut off in the middle
missing_meter_id / missing_transformer_id	The ID field is empty
bad_meter_id_format / bad_transformer_id_format	An ID that doesn't look like M-000000 or T-0000
missing_or_bad_ts	No time, or a time that can't be read
non_numeric_value	Text where a number should be, e.g. "kw":"abc"
negative_kw	Power below zero, which is impossible for a home

Table 12: The rules that send a reading to the dead-letter queue

Why `export/` exists: Snowflake loads plain Parquet files easily but does not understand Delta's change log. If it read Silver directly it could pick up old files that Delta has already replaced, and duplicates would come back. So each batch writes only its new clean rows to export/, and Snowflake loads each file exactly once.

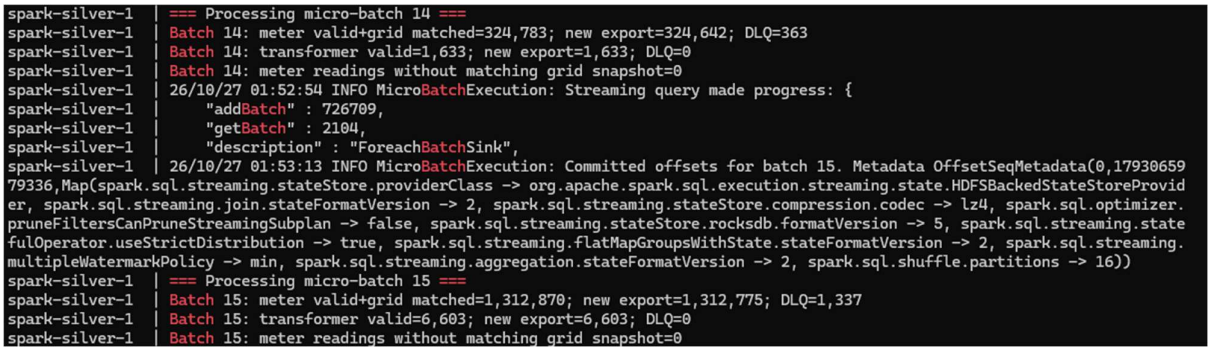


Figure 4: Silver batch log: 1,312,870 clean meter readings matched to the grid, 1,337 sent to the DLQ, 0 meters without a transformer

What went wrong and how we fixed it: at first, every batch compared its rows against the whole Silver table to find duplicates, so batches slowed from about 3 to about 20 minutes as Silver grew. We changed it to compare only against the same day's data (partition pruning), which kept batches fast.

Result: 9,591,495 clean meter readings and 48,000 transformer readings, with no duplicate keys, and every rejected row kept in dlq/ with its reason.

Phase 5 · Load into Snowflake

Owner	Mohamed Samir (setup), Abdulrhman Mohamed (automation)
Tools	Snowflake COPY INTO, external stages, storage integration
Takes in	S3 export/, grid/, truth/
Produces	Snowflake tables in the RAW schema

Snowflake reads the bucket through a secure link (a **storage integration**) that uses a read-only AWS role instead of a password. Three **stages** point at export/, grid/ and truth/. One SQL file (03_load_raw.sql) loads the five RAW tables. The day of each grid file is taken from its file name, which the history tracking needs.

> **In simple words:** COPY INTO keeps a list of the files it has already loaded. Running it again only loads new files, so running it twice never doubles the data.

RAW table	Rows after the initial load
METER_READINGS	9,591,495
TRANSFORMER_READINGS	48,000 (500 transformers × 96 slots)
GRID_METERS	200,000 (100,000 meters × 2 daily files)
GRID_TRANSFORMERS	1,000 (500 × 2 daily files)
SIM_TRUTH	815

Table 13: What the first load brought into Snowflake

Phase 6 · Gold: model, detect and test with dbt

Owner	Mohamed Samir
Tools	dbt (Snowflake adapter), dbt_utils
Takes in	The RAW tables
Produces	The CORE schema (history) and the MARTS schema (star schema, checks, KPIs, accuracy)

dbt lets us write each transformation as a SQL SELECT in its own file. dbt works out the right order from the references between files, builds each one as a table in Snowflake, tests the results and draws a lineage diagram of how everything connects. We used it for five jobs:

1. **History (snapshot):** records every change in the meter list day by day (SCD Type 2).
2. **Star schema:** two fact tables and two dimension tables (section 11).
3. **The three checks:** theft, overload and line drop, combined into one alert table, anomaly_flags (section 12).
4. **KPIs:** transformer peak loading, daily grid demand, alerts per type per day, and health per district.
5. **Accuracy:** compares anomaly_flags with the answer key and computes precision and recall.



Figure 5: dbt lineage graph (from dbt docs): green boxes are the RAW sources, blue boxes are the models built from them

Result: 14 models built, 1 snapshot, 15 tests passing; 985 alerts on day 1 (810 theft, 110 overload, 65 line drop) covering exactly the 25 planted transformers.

Phase 7 · Serve: the Power BI dashboard

Owner	Abdulrhman Mohamed
Tools	Power BI Desktop, Snowflake connector (Import mode)
Takes in	The MARTS tables
Produces	A 4-page dashboard (section 15)

Power BI connects with the read-only PBI_USER and imports the final tables. It shows the three KPIs the handbook asks for (transformer peak loading, anomaly flags, daily grid demand), a map of the alerts, a full alert log and the accuracy proof.

Phase 8 · Orchestrate: Airflow

Owner	Abdulrhman Mohamed
Tools	Apache Airflow 2.10.5
Takes in	Whatever is new in Bronze
Produces	Updated Silver, RAW and Gold tables every 15 minutes, with logs

Airflow runs the batch half of the pipeline in the right order every 15 minutes and keeps a log of every run. Details and the five required test runs are in section 13.

Following one reading through the whole pipeline

Stop	What the same reading looks like
Generator file	{"meter_id":"M-000865","ts":"2026-08-01T00:00:00Z","kw":0.552,"kvar":0.253,"voltage":223.34,"kwh":0.1379}
Kafka message	The same JSON text, in topic meter.readings
Bronze row	topic meter.readings · partition 0 · offset 10452 · arrival time · raw_value = the JSON above, untouched
Silver / export row	M-000865 · T-0004 · 2026-08-01 00:00 · 0.552 kW · 0.253 kvar · 223.34 V · 0.1379 kWh (now typed, checked, de-duplicated and tagged with its transformer)
RAW.METER_READINGS	The same row, inside Snowflake
MARTS.FACT_METER_READINGS	The same row, joinable to DIM_METERS and DIM_TRANSFORMERS
Theft check	Added to the sum of all homes on T-0004 for every 4-hour window, and compared with what T-0004 sent
Dashboard	Part of Total Demand; if T-0004 were stealing, it would appear on the map and in the alert log

Table 14: One meter reading, from creation to dashboard

A broken reading, by contrast

Stop	What happens
Generator file	{"meter_id":"M-000412", ... ,"kw":-0.9, ...}
Kafka and Bronze	Kept exactly as it is: Bronze never drops anything
Silver	Fails the rule "kw must not be negative" → written to dlq/meter_readings/ with reason negative_kw, its Kafka offset and the original text
Warehouse and dashboard	Never arrives, so it can't distort any total

10. The data layers: Bronze, Silver and Gold

We use the **medallion** pattern: data gets cleaner and more useful at each layer, and every layer is kept, so any number on the dashboard can be traced back to the raw message it came from.



Figure 6: What each layer contains

Place	Layer	Format	What it contains	Written by
S3 bronze/	Bronze	Parquet	Every Kafka message as it arrived, with topic, partition, offset and arrival time	Spark job 1
S3 silver/	Silver	Delta Lake, partitioned by date	Clean, typed, de-duplicated readings, each meter tagged with its transformer	Spark job 2
S3 dlq/	Silver (side)	Parquet, one folder per batch	Rejected rows, the reason, the original text and the Kafka position	Spark job 2
S3 export/	Silver (hand-off)	Parquet, partitioned by date and hour	Only the new clean rows of each batch, for Snowflake	Spark job 2
S3 grid/, truth/	Reference	CSV / JSON lines	Daily meter and transformer lists; the answer key	Run script
Snowflake RAW	Silver in the warehouse	Snowflake tables	Exact copies of export/, grid/ and truth/	COPY INTO
Snowflake CORE	Gold (history)	Snowflake table	METERS_SNAPSHOT: every version of every meter, with valid-from and valid-to dates	dbt snapshot
Snowflake MARTS	Gold	Snowflake tables	Facts, dimensions, the 3 checks, ANOMALY_FLAGS, KPIs, accuracy	dbt run

Table 15: Every storage place and what lives there

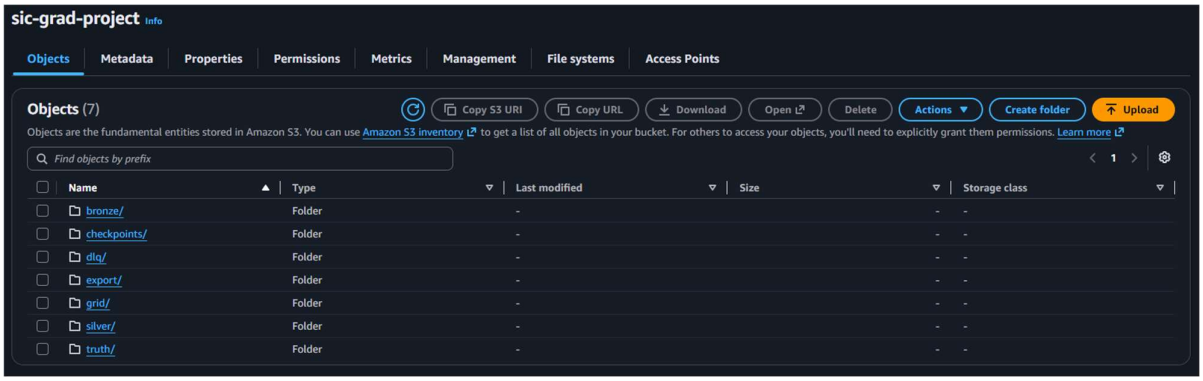
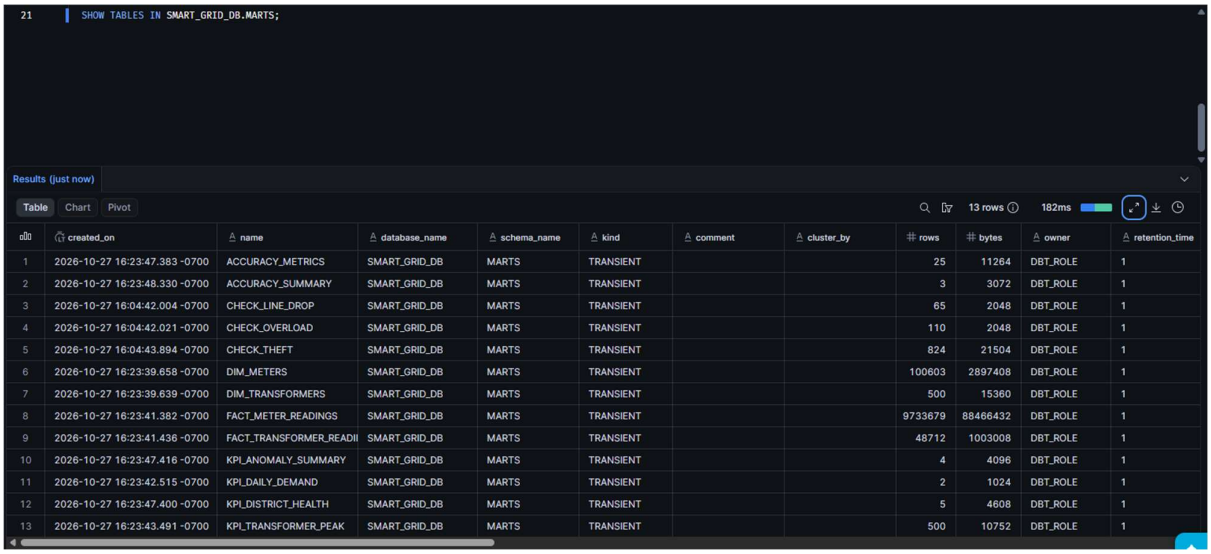


Figure 7: The data lake on S3: one folder per layer (checkpoints/ stores Spark's reading position)



	created_on	name	database_name	schema_name	kind	comment	cluster_by	rows	bytes	owner	retention_time
1	2026-10-27 16:23:47.383 -0700	ACCURACY_METRICS	SMART_GRID_DB	MARTS	TRANSIENT			25	11264	DBT_ROLE	1
2	2026-10-27 16:23:48.330 -0700	ACCURACY_SUMMARY	SMART_GRID_DB	MARTS	TRANSIENT			3	3072	DBT_ROLE	1
3	2026-10-27 16:04:42.004 -0700	CHECK_LINE_DROP	SMART_GRID_DB	MARTS	TRANSIENT			65	2048	DBT_ROLE	1
4	2026-10-27 16:04:42.021 -0700	CHECK_OVERLOAD	SMART_GRID_DB	MARTS	TRANSIENT			110	2048	DBT_ROLE	1
5	2026-10-27 16:04:43.894 -0700	CHECK_THEFT	SMART_GRID_DB	MARTS	TRANSIENT			824	21504	DBT_ROLE	1
6	2026-10-27 16:23:39.658 -0700	DIM_METERS	SMART_GRID_DB	MARTS	TRANSIENT			100603	2897408	DBT_ROLE	1
7	2026-10-27 16:23:39.639 -0700	DIM_TRANSFORMERS	SMART_GRID_DB	MARTS	TRANSIENT			500	15360	DBT_ROLE	1
8	2026-10-27 16:23:41.382 -0700	FACT_METER_READINGS	SMART_GRID_DB	MARTS	TRANSIENT			9733679	88466432	DBT_ROLE	1
9	2026-10-27 16:23:41.436 -0700	FACT_TRANSFORMER_READI	SMART_GRID_DB	MARTS	TRANSIENT			48712	1003008	DBT_ROLE	1
10	2026-10-27 16:23:47.416 -0700	KPI_ANOMALY_SUMMARY	SMART_GRID_DB	MARTS	TRANSIENT			4	4096	DBT_ROLE	1
11	2026-10-27 16:23:42.515 -0700	KPI_DAILY_DEMAND	SMART_GRID_DB	MARTS	TRANSIENT			2	1024	DBT_ROLE	1
12	2026-10-27 16:23:47.400 -0700	KPI_DISTRICT_HEALTH	SMART_GRID_DB	MARTS	TRANSIENT			5	4608	DBT_ROLE	1
13	2026-10-27 16:23:43.491 -0700	KPI_TRANSFORMER_PEAK	SMART_GRID_DB	MARTS	TRANSIENT			500	10752	DBT_ROLE	1

Figure 8: The Gold layer in Snowflake: 13 tables built by dbt (row counts taken during the demo run)

Why Delta Lake for Silver

Delta Lake turns a folder of Parquet files into a real table by adding a small change log (_delta_log/). That gives us what the handbook asks for on the lake side:

- **ACID writes:** a batch is either fully written or not at all, so a crash never leaves half-written data.
- **Schema enforcement:** rows with the wrong columns or types are refused.
- **MERGE (upsert):** update a row if its key exists, insert it if not, which is how Silver stays free of duplicates even when a batch is retried.
- **Time travel:** read the table as it was at an earlier version.
- **Partition pruning:** the MERGE only touches the dates in the current batch, not the whole table.

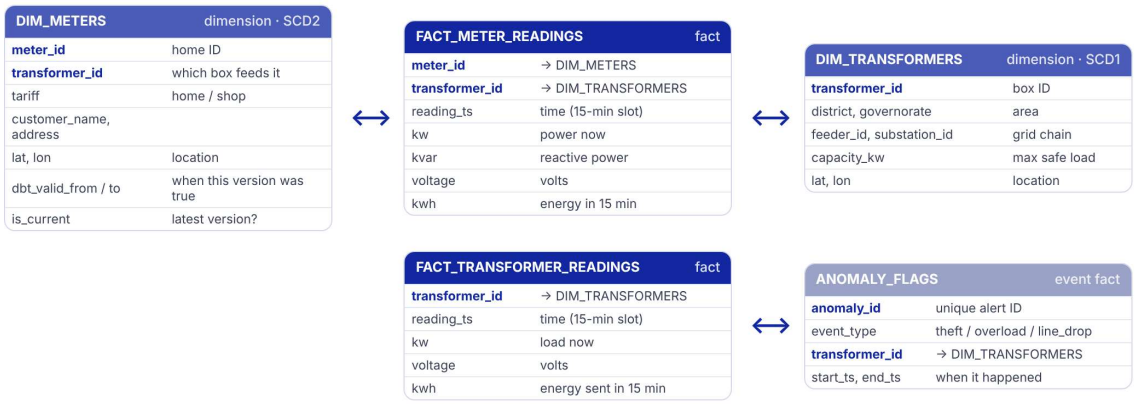
Snowflake has the same features built in. For example, its time travel let us count the readings as they were two hours earlier, before the demo data arrived:



Figure 9: Snowflake time travel: 9,733,679 meter readings now vs 9,691,495 two hours earlier

11. The data model

The Gold layer is a **star schema**: fact tables hold the measurements, and dimension tables describe them. This shape makes questions like "total energy per district per day" a simple join and a sum.



Blue keys join the tables. Facts hold the measurements (one row per meter or transformer per 15 minutes). Dimensions describe who and where. ANOMALY_FLAGS holds one row per detected problem.

Figure 10: The star schema in the Gold layer

Facts and their grain

The **grain** is what one row means. Stating it prevents double counting.

Fact table	Grain (one row is…)	Rows	Measures
FACT_METER_READINGS	one meter in one 15-minute slot	about 9.6 million per day	kw, kvar, voltage, kwh
FACT_TRANSFORMER_READINGS	one transformer in one 15-minute slot	48,000 per day	kw, voltage, kwh
ANOMALY_FLAGS	one detected problem window on one transformer	985 for day 1	count of alerts

Table 16: Fact tables and their grain

Dimensions and history (slowly changing dimensions)

Dimension	Describes	How changes are handled
DIM_METERS	Each meter: its transformer, tariff, customer, address, location	SCD Type 2: when a meter moves to another transformer or changes tariff, the old row is closed (it gets a valid-to date) and a new row is added. History is never lost. is_current marks the latest version
DIM_TRANSFORMERS	Each transformer: district, governorate, feeder, substation, capacity, location	SCD Type 1: we keep only the latest version, because transformers don't change between days

Table 17: Dimensions and how they keep history



```
SELECT meter_id, transformer_id, tariff, dbt_valid_from, dbt_valid_to
FROM SMART_GRID_DB.CORE.METERS_SNAPSHOT
WHERE meter_id IN (SELECT meter_id FROM SMART_GRID_DB.CORE.METERS_SNAPSHOT
                   GROUP BY meter_id HAVING COUNT(*) > 1)
ORDER BY meter_id, dbt_valid_from
LIMIT 10;
```

Results (just now) — Table | Chart | Pivot — 10 rows — 1.2s

	METER_ID	TRANSFORMER_ID	TARIFF	DBT_VALID_FROM	DBT_VALID_TO
1	M-000077	T-0001	residential	2026-10-27 03:34:07.135	2026-10-27 03:34:26.816
2	M-000077	T-0001	commercial	2026-10-27 03:34:26.816	null
3	M-000156	T-0001	residential	2026-10-27 03:34:07.135	2026-10-27 03:34:26.816
4	M-000156	T-0001	commercial	2026-10-27 03:34:26.816	null
5	M-000425	T-0003	residential	2026-10-27 03:34:07.135	2026-10-27 03:34:26.816
6	M-000425	T-0003	commercial	2026-10-27 03:34:26.816	null
7	M-000850	T-0005	residential	2026-10-27 03:34:07.135	2026-10-27 03:34:26.816
8	M-000850	T-0005	commercial	2026-10-27 03:34:26.816	null
9	M-001033	T-0006	residential	2026-10-27 03:34:07.135	2026-10-27 03:34:26.816
10	M-001033	T-0032	residential	2026-10-27 03:34:26.816	null

Figure 11: SCD Type 2 in Snowflake: meter M-000077 changed from residential to commercial, and M-001033 moved from transformer T-0006 to T-0032. The old version keeps its dates; the new one is open (valid-to is null)

Each meter reading also carries the transformer the meter belonged to **on the day of the reading** (added in Silver from that day's grid list). So if a meter moves, its old readings stay with the old transformer. That keeps the theft check correct over time.

Which numbers can be added up

Not every number can simply be summed. The handbook asks us to state this, and our KPIs follow these rules:

Number	Type	Rule we follow
kWh (energy)	Additive	Can be summed across meters, transformers and time. Daily demand = sum of kWh
kW (power)	Semi-additive	Can be summed across transformers at the same moment (network load), but not over time. Over time we take the maximum (peak) or the average
Voltage	Non-additive	Never summed. We look at the minimum or the average
Load %	Non-additive	Recomputed from kW ÷ capacity, never averaged from other percentages
Alert count	Additive	Can be summed. The number of distinct transformers is not additive (we use a distinct count)
Precision, recall	Non-additive	Recomputed from the totals of found, false and missed, never averaged

Table 18: Additive, semi-additive and non-additive measures

Power or energy? Units on the dashboard

Dashboard number	Power or energy	How it is calculated
Total Demand (GWh)	Energy	Sum of the kWh every transformer sent out, per day (includes normal cable loss), ÷ 1,000,000
Network Peak (MW)	Power	For each 15-minute slot, add up all transformers' kW; take the highest slot of the day; ÷ 1,000
Peak Load % (bar chart)	Power ÷ capacity	Each transformer's highest kW of the day ÷ its capacity × 100
Alert counts	Count	Rows in ANOMALY_FLAGS; each row is one flagged time window

Table 19: What each number on the dashboard means

Keys

Tables join on the business keys meter_id and transformer_id. The meter history table also gives each version its own unique version ID and validity dates, which works as the version key for SCD Type 2. Proper numeric surrogate keys on every dimension would be the next improvement (section 19).

12. The threats we catch and how we recognise them

All three checks are dbt models that run in Snowflake on the same clean data, at the end of the same pipeline. They only look at the readings. They never look at the answer key. Each check writes the time windows it flags into one table, anomaly_flags.

Threat	Data it uses	The rule	Time window
Overload	Transformer kw and its capacity_kw	Load is 85% of capacity or more in two readings in a row	30 minutes
Theft	Transformer kwh vs the sum of its homes' meter kwh	More than 11% of the energy is missing, and at least 95% of the expected meter readings arrived	4 hours, rolling
Line drop	Transformer voltage	Voltage below 207 V in four readings in a row	1 hour

Table 20: The three detection rules

12.1 Overload

What it is: a transformer is like a pipe of fixed size. When too many homes use heavy appliances at once, the load passes its safe limit. It overheats, wears out quickly, and can trip, cutting power to the whole street.

How we see it: load % = the transformer's power now (kW) ÷ its capacity (kW). Example: 412 kW ÷ 450 kW = 92%, close to tripping.

Why these numbers: a normal evening peak is about 75%. 85% is a warning line that leaves time to act before 100%. We ask for **two readings in a row** because one spike can be a kettle or a short burst, while 30 minutes in a row is a real trend.

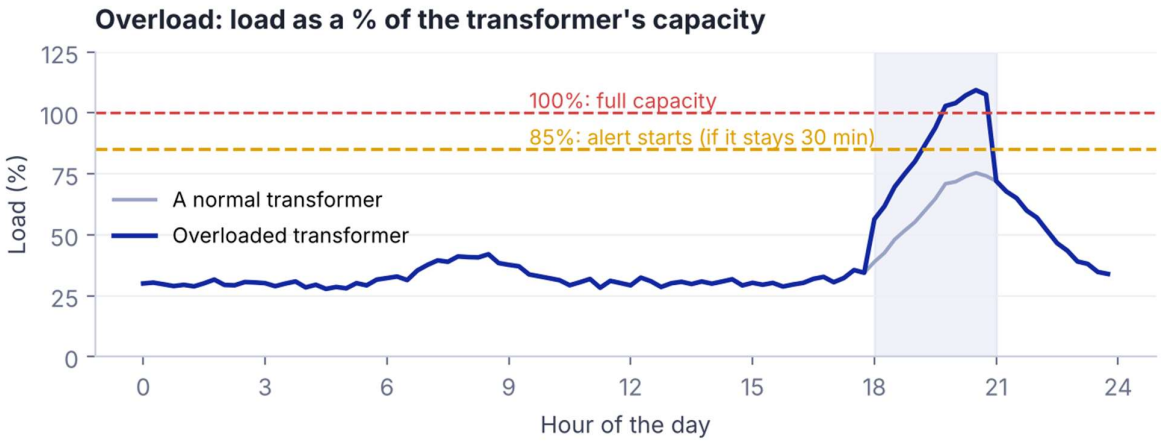


Figure 12: Illustration: an overloaded transformer climbs above the 85% line in the evening, while a normal one stays near 75%

12.2 Theft

What it is: someone bypasses or tampers with a meter, so it records only part of what the home really uses. The home still gets the electricity; the company doesn't get paid for it.

How we see it: the transformer measures everything it sends out, including stolen energy. The meters only record what passes through them. So for each transformer we compare: energy sent – energy recorded by all its homes = missing energy.

```
Transformer sent:    500 kWh  (last 4 hours)
Homes recorded:      410 kWh
Missing:             90 kWh  = 18%  →  more than 11%  →  theft suspected
```

Why 11%: about 6% is always lost as heat in the cables, even with no theft. We only flag when the gap is clearly bigger: 6% normal loss + a 5% safety margin. On test data, a 5% line flagged honest transformers (false alarms), and a 20% line missed a real thief.

Why the 95% rule: a meter that stops sending (a silent meter) looks exactly like stolen energy, because its usage disappears from the sum. We only judge a window when at least 95% of the expected readings arrived. **Missing data must never become an accusation.** That is why our 500 planted silent meters caused zero false alarms.

Why 4 hours: a 4-hour rolling window smooths out the random ups and downs of single readings, so only a steady gap counts.

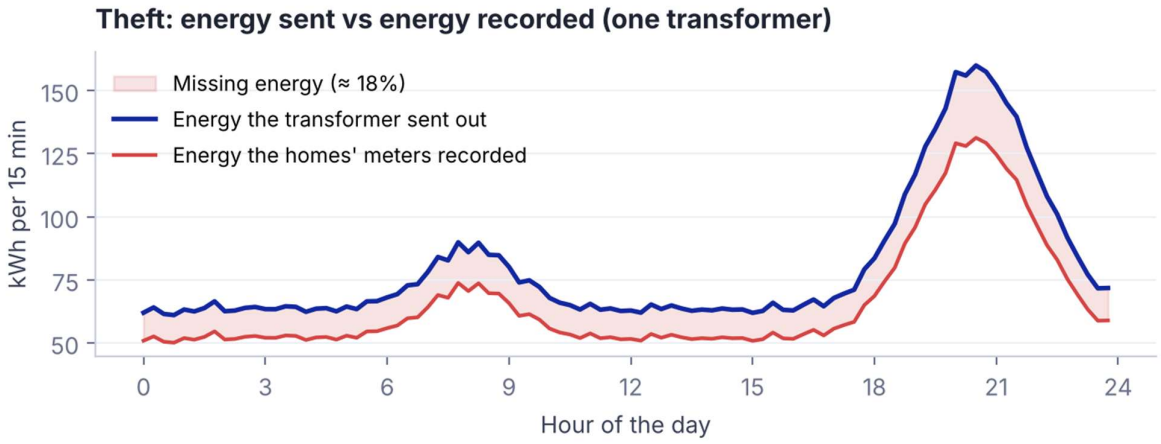


Figure 13: Illustration: on a transformer with thieves, the energy it sends stays well above what its meters record

12.3 Line drop

What it is: voltage is the electrical pressure. A damaged, loose or overloaded cable makes it fall. Lights dim, motors struggle, and appliances can be damaged.

How we see it: the transformer's voltage reading stays below 207 V for four readings in a row.

Why 207 V: it is 10% below 230 V, the usual lower limit for a normal supply. We ask for **one hour** in a row so a short dip doesn't raise an alarm; a damaged cable keeps the voltage low.

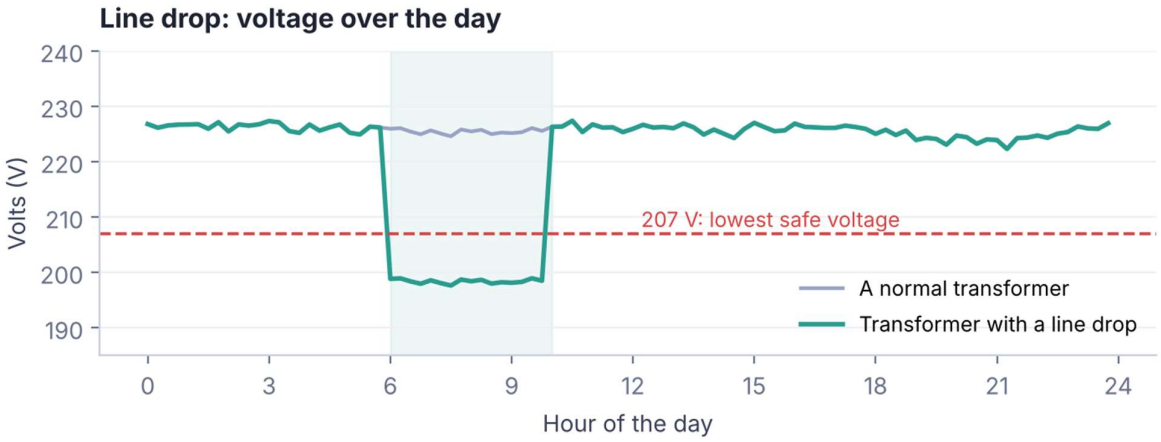


Figure 14: Illustration: during a line drop the voltage falls to about 198 V and stays below the 207 V line

12.4 Why one problem creates many alerts

Each alert is one **flagged time window**, not one incident. A thief steals all day, so a theft transformer is flagged in every 4-hour window of the day: 81 windows × 10 transformers = 810 theft alerts. An overload lasting 3 hours gives 11 windows × 10 = 110. A 4-hour line drop gives 13 windows × 5 = 65. Together: 985 alerts on day 1, from exactly 25 transformers. This is why the dashboard shows both the number of alerts and the number of transformers.

> **In simple words:** the flags show patterns that need an inspection. They are not proof that someone stole electricity, and on our project they describe simulated data only.

13. Orchestration with Airflow

Airflow is the pipeline's alarm clock and manager. A pipeline in Airflow is called a **DAG**: a list of tasks and the order they must run in. Ours is called smart_grid_pipeline.



Figure 15: What Airflow manages, and what runs on its own

What Airflow manages, and what it doesn't

Airflow manages the **batch** half: cleaning what is new in Bronze, loading Snowflake, and the dbt steps. The **streaming** half (producer → Kafka → Bronze) is not run by Airflow, because streaming never stops and doesn't wait for a timer. It runs in Docker on its own and keeps filling Bronze. This split is normal in real systems.

The five tasks

Task	What it does	How
clean_bronze	Cleans whatever is new in Bronze into Silver and export/	Starts Spark job 2 in Docker in run-once mode; it processes the backlog and stops. If the continuous cleaning job is already running (live demo), this task skips itself so the two never clash
load_to_snowflake	Loads new export/, grid/ and truth/ files into RAW	Runs 03_load_raw.sql with the dbt login; only new files load
dbt_snapshot	Records meter changes (SCD2)	dbt snapshot
dbt_run	Rebuilds the star schema, the 3 checks and the KPIs	dbt run; the checks only process new time windows (incremental models)
dbt_test	Runs the data tests: the quality gate	dbt test; if any test fails, the run turns red

Table 21: The tasks of the smart_grid_pipeline DAG

DAG settings

Setting	Value	Why
Schedule	Every 15 minutes	Matches the meters' 15-minute reporting interval
Dependencies	Each task runs only if the one before succeeded	Bad data never reaches the dashboard
Retries	1 automatic retry after 2 minutes	Temporary problems (network, Snowflake waking up) fix themselves
Max active runs	1	Two runs never load or rebuild the same tables at the same time
Catch-up	Off	Turning it on doesn't start hundreds of old runs
Logs	Kept for every task of every run	Proof the pipeline ran, and the first place to look when something fails

The five required batch runs

The handbook's data-volume guide asks for at least five batch runs that prove the pipeline is correct, idempotent (running twice doesn't change the result) and recoverable. We ran all five:

#	Run	What we did	What it proves	Result
1	Initial load	Loaded the full streamed day 1	The full history loads	9,591,495 meter / 48,000 transformer readings
2	Incremental	Sent one new 15-minute slot of day 2, then triggered the DAG	Only new data is processed	+100,000 meter and +500 transformer readings (100,613 sent; 613 duplicates and late copies removed)
3	Same run again	Triggered the DAG again straight away	No duplicates (idempotency)	"Copy executed with 0 files processed"; counts unchanged
4	Failure + retry	Forced dbt_run to fail mid-run, then cleared it	A failure stops the run; we rerun only the failed step	dbt_test was blocked (upstream failed); after Clear, both went green
5	Backfill	Rebuilt the 3 checks over all past data (full refresh)	Old data can be reprocessed correctly	3 models rebuilt: 65 line-drop, 110 overload, 820 theft windows

Table 22: The five batch runs and their results



Figure 16: Run 2 → 3: counts after the new slot (9,691,495 / 48,500), unchanged after running again

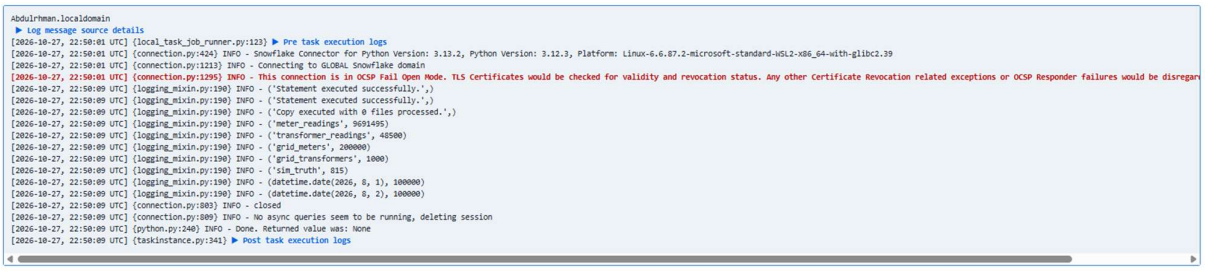


Figure 17: Run 3: the load task found 0 new files, so nothing was loaded twice

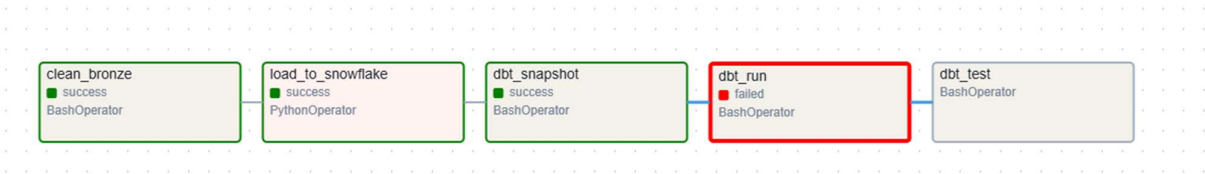


Figure 18: Run 4: dbt_run forced to fail; dbt_test is held back (upstream_failed)

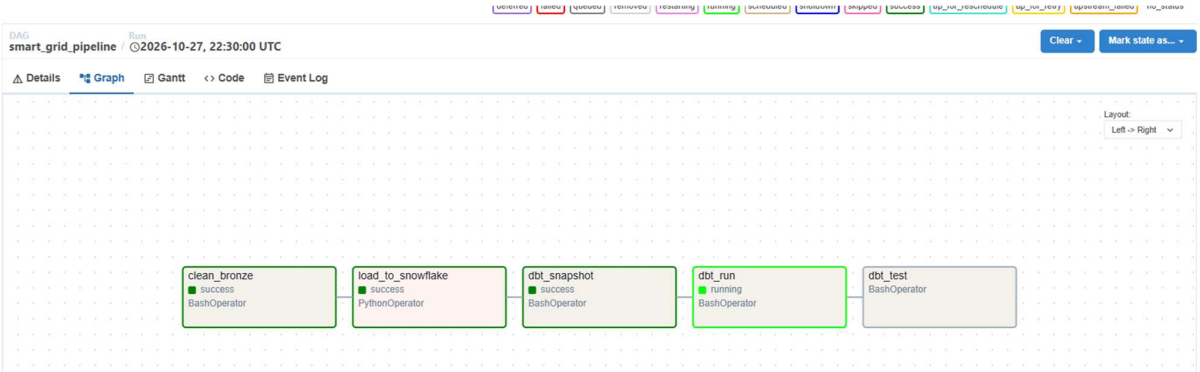


Figure 19: Run 4: after Clear, dbt_run runs again

```
(base) yeager@Abdulrhman:/mnt/c/Users/moham$ cd ~/smart_grid_pipeline/dbt
(base) yeager@Abdulrhman:~/smart_grid_pipeline/dbt$ .venv/bin/dbt run --full-refresh --select check_theft check_overload check_line
_drop
23:04:35  Running with dbt=1.13.0-b1
23:04:35  Registered adapter: snowflake=1.12.4
23:04:36  Found 14 models, 1 snapshot, 16 data tests, 5 sources, 679 macros
23:04:36
23:04:36  Concurrency: 4 threads (target='dev')
23:04:36
23:04:41  1 of 3 START sql incremental model MARTS.check_line_drop ....................... [RUN]
23:04:41  2 of 3 START sql incremental model MARTS.check_overload ........................ [RUN]
23:04:41  3 of 3 START sql incremental model MARTS.check_theft ........................... [RUN]
23:04:43  1 of 3 OK created sql incremental model MARTS.check_line_drop .................. [SUCCESS 65 in 1.75s]
23:04:43  2 of 3 OK created sql incremental model MARTS.check_overload ................... [SUCCESS 110 in 1.75s]
23:04:46  3 of 3 OK created sql incremental model MARTS.check_theft ...................... [SUCCESS 820 in 4.50s]
23:04:46
23:04:46  Finished running 3 incremental models in 0 hours 0 minutes and 10.67 seconds (10.67s).
23:04:46
23:04:46  Completed successfully
23:04:46
23:04:46  Done. PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=3
(base) yeager@Abdulrhman:~/smart_grid_pipeline/dbt$
```

Figure 20: Run 5: backfill of the three checks

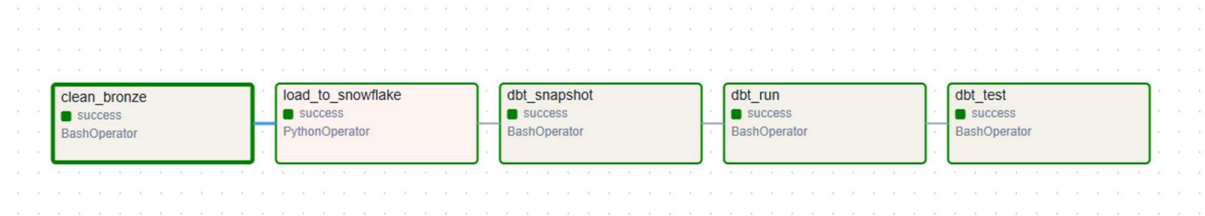


Figure 21: A complete successful run: all five tasks green

14. Data quality and testing

Quality is checked at every layer, not only at the end. Each layer catches a different kind of problem.

Where	What is checked	What happens to bad data
Producer	Kafka confirms every message (acks = all), with retries	Failed sends are counted and reported (0 in our runs)
Bronze	Nothing removed on purpose	Kept forever as the raw record, so any problem can be investigated
Silver	Valid JSON, required IDs, ID format, valid time, numbers are numbers, no negative power	Sent to dlq/ with the reason and the original text
Silver	Duplicates (same meter + same time)	Only one copy is kept (batch de-duplication + Delta MERGE)
Silver	Every meter must exist in that day's grid list	Counted and reported (0 unmatched in our runs)
Snowflake load	Each file is loaded only once	Re-running the load adds nothing
dbt tests	15 tests on the Gold tables (below)	A failure turns the Airflow run red and stops it
Accuracy model	Alerts compared with the answer key	Shows found, false alarms and missed for each threat

Table 23: Quality checks at every layer

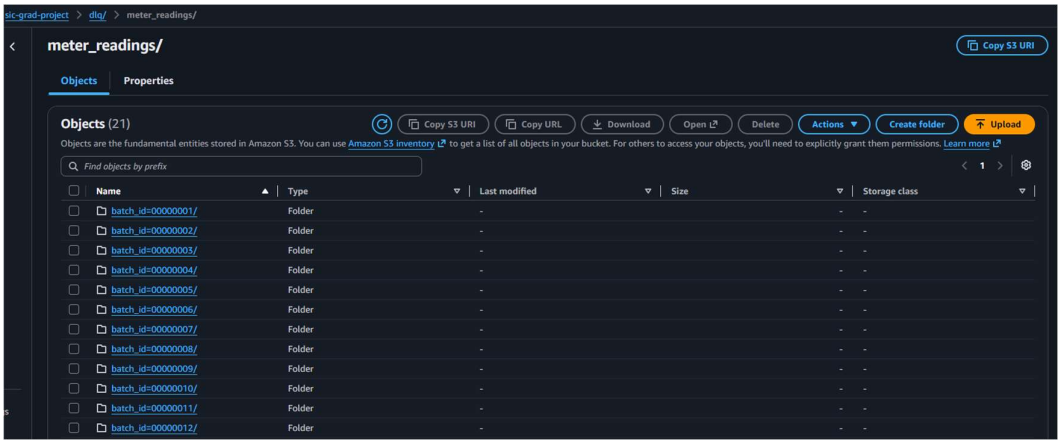


Figure 22: The dead-letter queue on S3: one folder per Spark batch with the rejected rows\

The dbt tests

Table	Tests
DIM_TRANSFORMERS	transformer_id is unique and never empty; capacity_kw never empty
DIM_METERS	meter_id never empty; every transformer_id exists in DIM_TRANSFORMERS
FACT_TRANSFORMER_READINGS	transformer_id and reading_ts never empty
FACT_METER_READINGS	meter_id and kw never empty; meter + time is unique (warning level)
ANOMALY_FLAGS	anomaly_id unique and never empty; event_type is only theft, overload or line_drop; every transformer exists
ACCURACY_METRICS	detection_status is only True Positive, False Positive or False Negative

Table 24: The 15 automated data tests

The quality gate, demonstrated

We also wrote one test that **fails on purpose**: "no transformer may go above 100% of its capacity". The planted overloads break this rule, so when we run it the test fails and the pipeline stops. This shows the gate really blocks bad results. Normal runs skip it (`dbt test --exclude tag:demo`); the demo runs it alone (`dbt test --select tag:demo`).

Accuracy against the answer key

The model `accuracy_metrics` lines up the planted problems (from `sim_truth`) with the detected ones (from `anomaly_flags`), per transformer and threat type, and labels each pair:

- **True Positive:** planted and found.
- **False Positive (false alarm):** found, but nothing was planted there.
- **False Negative (missed):** planted, but not found.

Then `accuracy_summary` computes **precision** (of everything we flagged, how much was real) and **recall** (of everything planted, how much we caught).

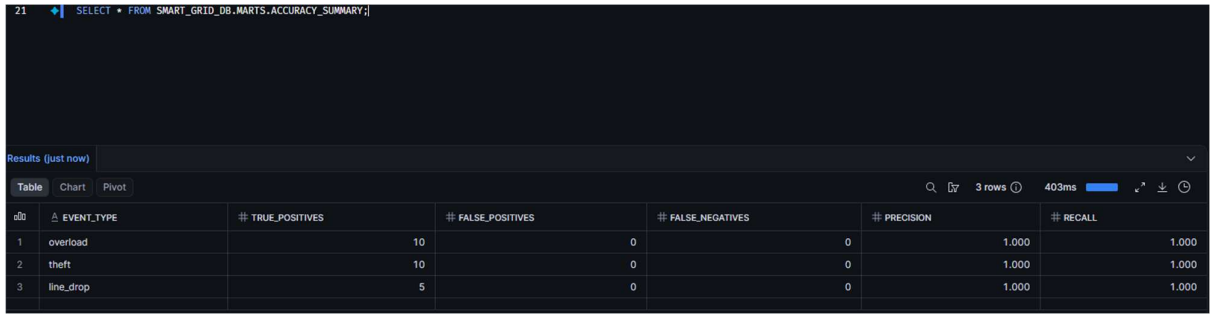


Figure 23: Accuracy: 10 overload, 10 theft and 5 line-drop transformers found, 0 false alarms, 0 missed

15. The Power BI dashboard

15.1 Connection and model

Power BI connects to Snowflake with the read-only user PBI_USER and **imports** seven tables from the MARTS schema: the four KPI tables, ANOMALY_FLAGS, ACCURACY_SUMMARY and DIM_TRANSFORMERS. Import mode keeps the dashboard fast and doesn't wake Snowflake (and use credits) on every click. The cost is that it shows the data as of the last refresh, so we press Refresh after an Airflow run.

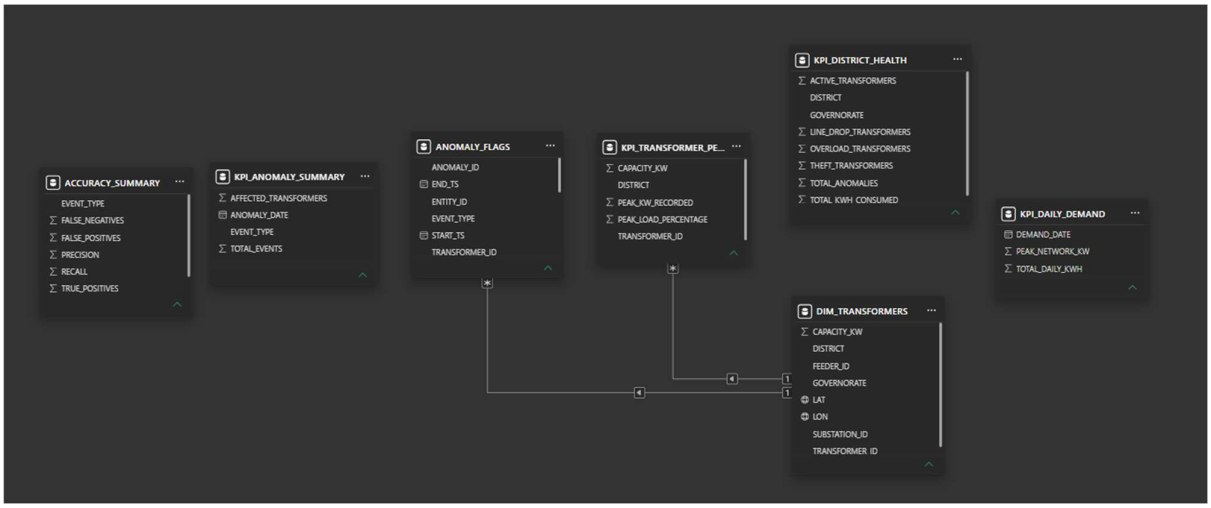


Figure 24: The Power BI model: DIM_TRANSFORMERS links to the alerts and to the peak-load table

From	To	Type	Why
DIM_TRANSFORMERS[TRANSFORMER_ID]	ANOMALY_FLAGS[TRANSFORMER_ID]	One to many, single direction	Gives every alert its district and map location
DIM_TRANSFORMERS[TRANSFORMER_ID]	KPI_TRANSFORMER_PEAK[TRANSFORMER_ID]	One to many, single direction	Lets the district filter also filter peak loads

Table 25: Relationships in the Power BI model

The other tables are already summarised by dbt, so they stand alone. We switched off Power BI's automatic relationship detection because it guessed wrong links between unrelated columns with the same name.

15.2 Measures (DAX)

Measure	Formula	Shown on
Alerts	COUNTROWS(ANOMALY_FLAGS)	Map bubble size, donut, tables
Theft / Overload / Line Drop Transformers	CALCULATE(DISTINCTCOUNT(ANOMALY_FLAGS[TRANSFORMER_ID]), ANOMALY_FLAGS[EVENT_TYPE] = "theft") (same for the other two)	Overview, Map, Log cards
Flagged Transformers	DISTINCTCOUNT(ANOMALY_FLAGS[TRANSFORMER_ID])	Map, Log cards
Total Demand	FORMAT(DIVIDE(SUM(KPI_DAILY_DEMAND[TOTAL_DAILY_KWH]), 1000000), "#,##0.00") & " GWh"	Overview card
Network Peak	FORMAT(DIVIDE(MAX(KPI_DAILY_DEMAND[PEAK_NETWORK_KW]), 1000), "#,##0.0") & " MW"	Overview card
Detected / False Alarms / Missed	SUM(ACCURACY_SUMMARY[TRUE_POSITIVES]) (and FALSE_POSITIVES, FALSE_NEGATIVES)	Accuracy cards
Planted	SUM(...[TRUE_POSITIVES]) + SUM(...[FALSE_NEGATIVES])	Accuracy chart
Precision %	FORMAT(DIVIDE(TP, TP + FP), "0.0%")	Accuracy card
Recall %	FORMAT(DIVIDE(TP, TP + FN), "0.0%")	Accuracy card

Table 26: The DAX measures

Precision and recall are recomputed from the totals instead of averaging the per-type values, because percentages can't be averaged correctly.

15.3 The design

The report uses a dark control-room style in the project blue (#1328A0), like the screens in a real grid operations centre. Each threat has one colour everywhere: **red for theft**, **amber for overload** and **teal for line drop**, so a viewer links the cards, charts and map without reading labels. The header shows the page name and the pipeline steps.

15.4 How to read each page

Page 1 · Overview



Figure 25: Overview page

- **Theft, Overload, Line Drop Transformers:** how many transformers have at least one alert of that type (10, 10, 5).
- **Total Demand:** all the electricity the grid delivered over the loaded days, in gigawatt-hours (energy).
- **Network Peak:** the highest total load on the whole grid at one moment, in megawatts (power).
- **Top 10 Transformers by Peak Load %:** each bar is a transformer's highest load compared with its capacity. The amber line (85%) is where alerts start; the red line (100%) is full capacity. All ten bars sit at 109–111%: these are the overloaded transformers.
- **Alert Events by Type:** all flagged time windows split by type. Theft is the largest slice because thieves steal all day (section 12.4).

Page 2 · Alert Map

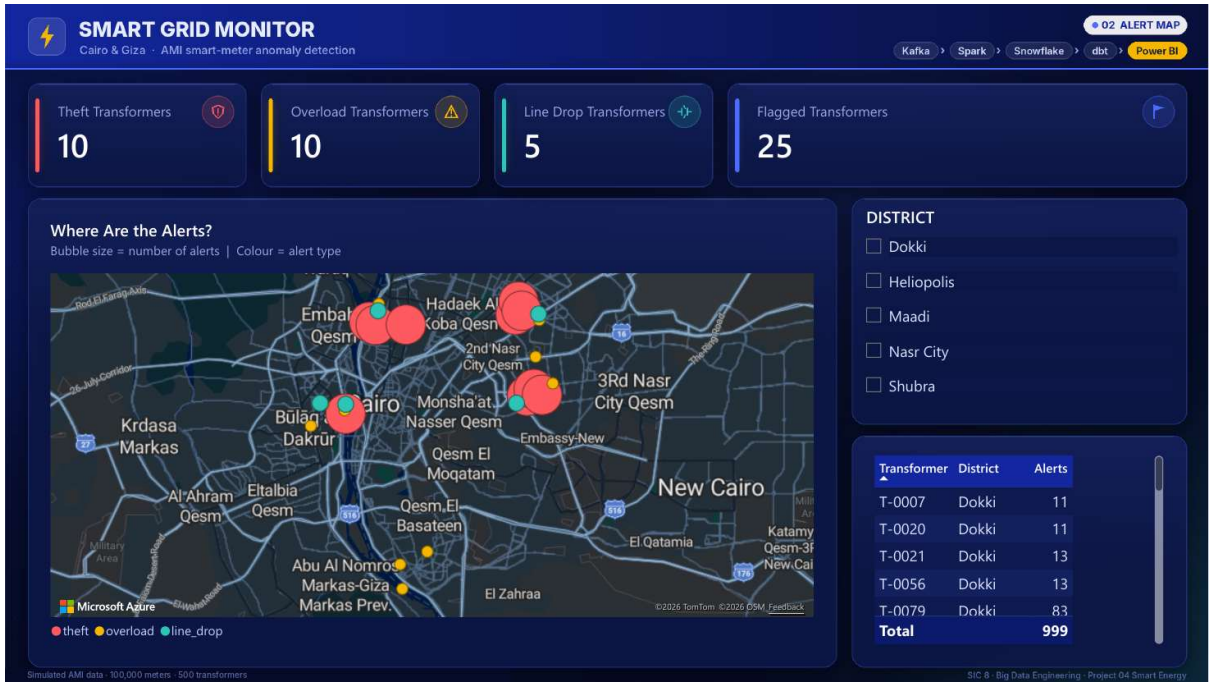


Figure 26: Alert Map page

- Each bubble is a transformer at its real location. **Bigger bubble = more alerts; colour = threat type.**
- The **District** filter on the right narrows everything on the page to one area.
- The table ranks transformers by number of alerts, so a supervisor knows where to send a crew first.

Page 3 · Alert Log

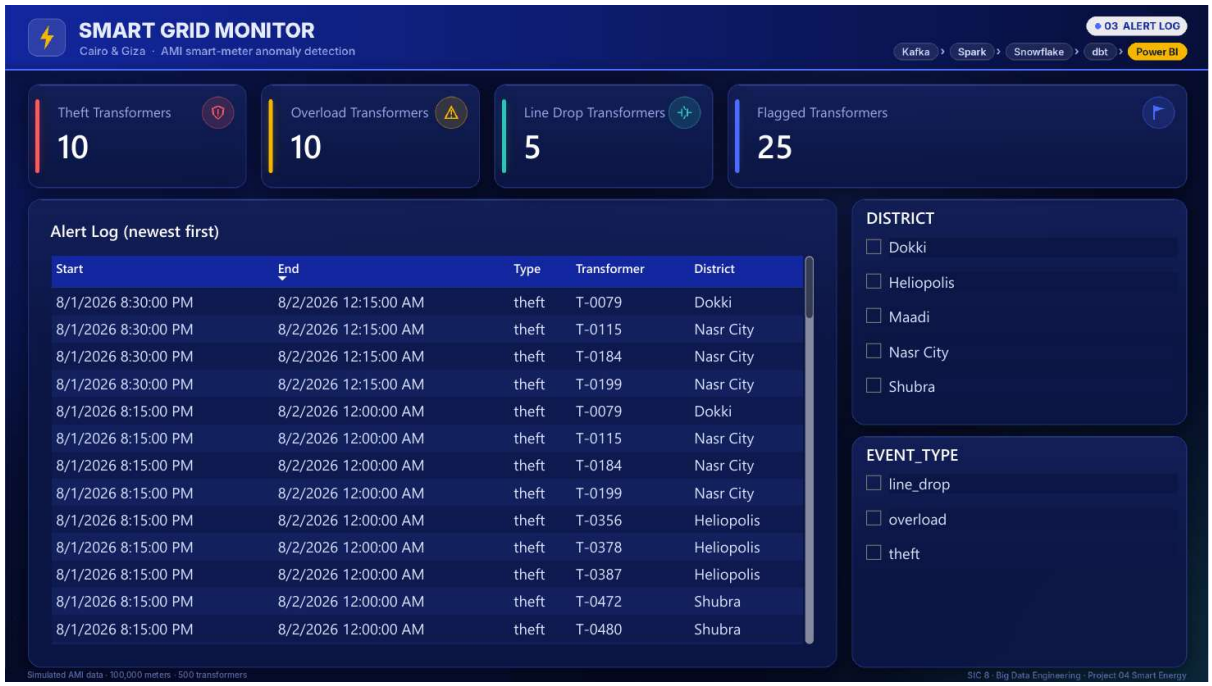


Figure 27: Alert Log page

- Every alert with its start time, end time, type, transformer and district, **newest first**.

- Filter by district or by type. This is the working list an engineer would act on.

Page 4 · Accuracy

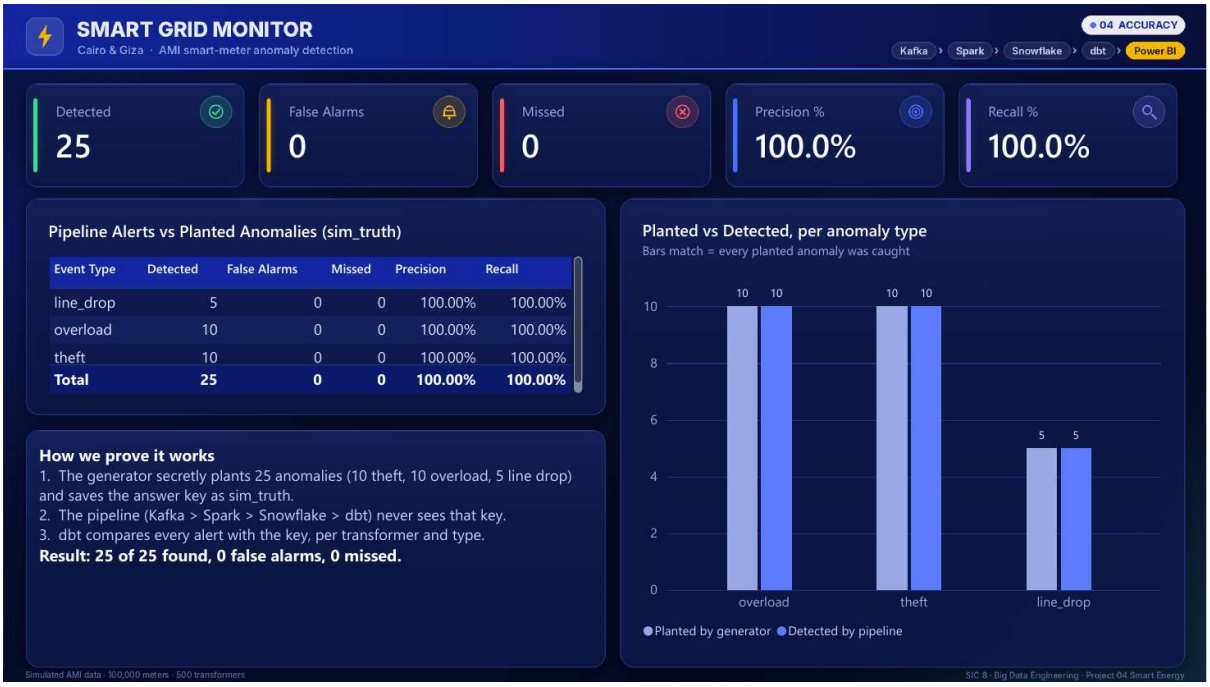


Figure 28: Accuracy page

- **Detected 25, False Alarms 0, Missed 0, Precision 100%, Recall 100%.**
- The table shows the same per threat type.
- **Planted vs Detected:** for each type, the light bar (planted by the generator) and the blue bar (detected by the pipeline) are the same height.
- The text box explains how we test without cheating: the pipeline never sees the answer key.

16. Results and measured numbers

wall-clock time (how long the run really took) from **event time** (the simulated time the data represents).

Measure	Value
Event time streamed	1 full day (1 Aug 2026, 00:00–24:00) = 96 windows of 15 minutes
Wall-clock time of the stream	About 80 minutes, so the simulated day ran about 18 times faster than real time
Streaming rate	2,000 meter messages per second + 10 transformer messages per second
Messages sent to Kafka	9,649,221 meter + 48,000 transformer, 0 failed
Clean meter readings (Silver → RAW)	9,591,495 (expected 9.6 million; the gap is the 500 planted silent meters and a few late readings that fall into the next day's file)
Removed as duplicates or broken	About 57,700 messages (≈ 0.6%), matching the planted 0.5% duplicates + 0.1% broken
Silver lag behind the stream	10–20 minutes on the laptop (limited by upload speed, about 0.65 Mbps)
Grid history (SCD2)	100,000 meters on day 1; 603 changed meters added as new versions on day 2
dbt	14 models + 1 snapshot built; 15 tests passing
Alerts on day 1	985 (810 theft, 110 overload, 65 line drop) on exactly 25 transformers
Accuracy	25 / 25 found, 0 false alarms, 0 missed
Incremental run (one 15-minute slot)	100,613 messages sent in about 55 seconds → +100,000 clean meter rows, +500 transformer rows
Day-1 demand and peak	About 4.31 GWh delivered; network peak 316.8 MW

Table 27: Measured results

Against the handbook's data-volume guide (Energy / AMI)

Guide asks for	We did
Streaming functional run: 1–2M readings in 60 minutes, crossing 4–5 windows of 15 minutes	9.65M readings in about 80 minutes, crossing 96 windows
Batch initial load: 9.6M readings (1 day)	9,591,495 clean readings loaded
Hourly incremental: about 400k readings	15-minute slots of about 100k readings, processed incrementally by Airflow
At least 5 batch runs	Initial load, incremental, same run again, failure + retry, backfill
Excellent level extras: restart, late events, SCD, quality-gate failure demo	Restart and recovery from Kafka, 1% late readings, SCD2 snapshot, a test that fails on purpose

Table 28: Our run compared with the data-volume guide

17. Key decisions and why we made them

Decision	Options we had	What we chose and why
The idea	Banking, healthcare, fleet, smart meters, e-commerce	Smart meters with three threats (theft, overload, line drop). Electricity theft and failing transformers are real, costly problems, and the idea goes beyond a plain pipeline to a monitoring system
Stream and batch layout	Day 1 as a batch file + day 2 as a stream, or everything through the stream	Everything through Kafka. One code path is simpler to build, test and explain. The batch side is still there: daily grid files and the 15-minute Airflow loads
Where to run Spark	AWS Glue / EMR, or local Docker	Local Docker. Free, fast to debug, no console access needed; same code can move to the cloud
Cloud accounts	Everyone uses their own, or one shared owner	One owner, one bucket, one key per person. Follows our instructor's advice not to share accounts; a leaked key affects only one user
Bronze format	Delta, or plain Parquet	Plain Parquet with the message untouched. Bronze is append-only and never updated, so Delta's features aren't needed there
Silver format	Plain Parquet, Iceberg, Delta	Delta Lake. ACID, MERGE and time travel, and no separate catalog service (Iceberg needs one)
Hand-off to the warehouse	Snowflake reads Silver directly, or a separate export folder	A separate export/ folder. Snowflake doesn't read Delta's log, so reading Silver could bring back replaced files and duplicates
Warehouse	Snowflake, Databricks, Athena	Snowflake. Built-in loading from S3, fast SQL, time travel, trial credits; Databricks Free Edition couldn't read our own bucket
Loading	Snowpipe, Airbyte, COPY INTO	COPY INTO, scheduled by Airflow. Free, simple, and never loads a file twice
Transformations	Stored procedures / SQL scripts, or dbt	dbt. Ordering, testing, history (snapshots) and lineage documentation in one tool
Thresholds	Stricter or looser rules	85% for 30 min, 11% over 4 h with 95% data, 207 V for 1 h. Each is set above normal behaviour with a safety margin (section 12)
Keep the answer key separate	Use the planted labels in the rules, or keep them out	Keep them out. The handbook warns that labels must not leak into detection rules; we only use them to score accuracy
Power BI mode	DirectQuery (live) or Import	Import. Faster pages and no Snowflake credits used on every click; refreshed after each run
Final run	Spread across members' accounts, or one person's accounts	One person's accounts end to end. The whole run is reproducible without depending on anyone else's access

Table 29: Key decisions

18. Problems we hit and how we fixed them

A working pipeline is mostly about fixing what doesn't fit. These are the real problems from building and joining the four parts, and how each was solved.

Problem	Cause	Fix
Access denied when uploading to S3	The bucket policy named an example bucket instead of ours	Corrected the bucket name in the policy
Snowflake couldn't use the AWS role	Our Snowflake account is in Zurich, a region AWS keeps switched off by default	Enabled the Zurich region and its security-token endpoint in AWS
Every row went to the DLQ in a test	Spark job 1 named the message column value, Spark job 2 expected raw_value	Aligned the column name
Spark job 2 saw no data	Job 1 writes two sub-folders; job 2 read only the top folder	Read both sub-folders
Overload and line drop never appeared	The first stream mode sent only the first 5 hours of the day	Added a full-day mode that streams the whole day
First Spark start timed out	Spark downloads its add-ons on first start	Downloaded them once in advance and cached them
The first 20 seconds of readings missing	Spark started reading after the producer had begun	Recovered them from Kafka; the run script now waits for Spark
Silver batches slowed from 3 to 20 minutes	Each batch compared against all of Silver to find duplicates	Compare only against the same day (partition pruning)
Admin user couldn't read the Gold tables	The tables belong to custom roles not granted to the admin roles	Granted the dbt and Power BI roles to the system admin role
Power BI linked the wrong columns	Automatic relationship detection matched unrelated columns	Turned it off and created the two correct relationships by hand
Power BI refresh failed once	It refreshed while dbt was rebuilding a table	Refresh only after the Airflow run is green
Commands very slow or silent	Working inside the Windows drive from Ubuntu	Work inside the Ubuntu home folder

Table 30: Problems and fixes

19. Limitations and what we would do next

We want to be clear about what this project does not do yet.

Limitation	Why it matters	Next step
All processing runs on one laptop	Upload speed and memory limited throughput; Silver ran 10–20 minutes behind the stream	Run Spark on a cloud cluster (EMR or Databricks) next to the bucket
One Kafka broker, one partition per topic	No fault tolerance and limited parallel reading	Three brokers and several partitions per topic
One simulated day streamed in full (≈ 80 min)	Meets the recommended level; the optional 2–4 hour stress test was not run	A longer stress run at a higher rate
Theft is located per transformer, not per house	An inspector still has to find the house among about 200	Compare each meter with its own history and with similar homes
Thresholds tuned on our simulator	Real grids are noisier	Tune on real or historical data
Business keys instead of numeric surrogate keys	Works, but surrogate keys are the textbook star-schema design	Add surrogate keys to the dimensions
dbt tests run after the tables are rebuilt	A failed test stops the run, but the new tables already exist for a moment	Build into a staging schema and swap only after tests pass
Airflow in standalone mode (SQLite, one task at a time)	Fine for a laptop, not for production	Airflow with PostgreSQL and the Celery or Kubernetes executor
S3 in Stockholm, Snowflake in Zurich	Cross-region transfer costs a little and needed an extra setup step	Keep both in the same region
Power BI in Import mode	Shows data as of the last refresh	Scheduled refresh in the Power BI Service
Synthetic data	Results describe our simulation, not a real grid	Pilot on real anonymised meter data
Demo recorded as a video	Connectivity and runtime could disrupt a live run	Keep the live run as a bonus, the video as a fallback

Table 31: Limitations

20. How to run the project

The full step-by-step guide is RUN_GUIDE.md in the repository. In short:

1. **Install once:** Ubuntu on Windows (WSL 2), Docker Desktop with WSL integration, the AWS CLI and Python.
2. **Cloud setup once:** an AWS key for the bucket in .env; in Snowflake run warehouse/01_setup_my_snowflake.sql (warehouse, database, schemas, stages, users and roles) and link Snowflake to the AWS role.
3. **Build the containers:** docker compose --profile tools --profile silver build
4. **Smoke test:** ./m1/run_m1.sh smoke → wait for SMOKE PASS.
5. **Full run:** ./m1/run_m1.sh fullday in one terminal and docker compose --profile silver up spark-silver in another.
6. **Load and model:** run warehouse/03_load_raw.sql, then dbt snapshot, dbt run and dbt test --exclude tag:demo.
7. **Automate:** start Airflow (airflow standalone), switch on smart_grid_pipeline.
8. **New data:** ./m1/run_m1.sh demo 00:30 sends one new 15-minute slot; Airflow processes it.
9. **Dashboard:** open SIC_SMART_GRID_DARK.pbix and press Refresh.

Because the generator uses a fixed seed and one settings file, repeating these steps produces the same data and the same results.

21. How we meet requirements

This section maps every requirement in the graduation handbook to where it is met in our project.

Pillar 1 · Domain-specific data generation

Handbook requirement	How we meet it	Section
Custom Python generator with realistic schemas (JSON/CSV)	generator.py: JSON readings, CSV grid lists, JSON answer key	8
Concurrent generation (threading / asyncio)	The producer sends meter and transformer streams in parallel threads; the generator uses vectorised NumPy	9 · P2
Inject controlled domain anomalies	Overload, line drop, theft, plus silent meters, duplicates, broken and late messages	8.4
Real-time broker streaming and scheduled micro-batches	Kafka stream; Spark micro-batches; Airflow every 15 minutes	4, 13
Reproducible settings and anomaly controls; not a fixed CSV	One config.yaml with a fixed seed	8.5

Pillar 2 · Data ingestion

Handbook requirement	How we meet it	Section
Kafka for high-velocity topics	Kafka 3.9.1, topics meter.readings and transformer.readings	9 · P2
Scheduled landing	Daily grid files uploaded by the run script; Airflow loads every 15 minutes	9, 13
Immutable bronze/raw zone on S3	S3 bronze/: messages kept untouched with Kafka metadata	9 · P3, 10
Dead-letter queue	S3 dlq/ with reason, original text and Kafka position	9 · P4, 14
Ingestion logs and retry backoff	Producer logs and confirmations with 5 retries; Airflow retry after 2 minutes	9 · P2, 13
Failed events visible and recoverable	DLQ keeps the original message; missed messages were recovered from Kafka	9 · P3–P4

Pillar 3 · Distributed processing

Handbook requirement	How we meet it	Section
PySpark batch or Structured Streaming	Two Structured Streaming jobs (continuous or run-once)	9 · P3–P4
Enforce schemas, handle nulls	Fixed schema; safe number conversion; null and format checks	9 · P4
Deduplicate composite business keys	(meter_id, ts) and (transformer_id, ts), in-batch + Delta MERGE	9 · P4
Enrich events through lookups	Each meter reading joined with that day's grid list	9 · P4
Snappy-compressed Parquet with physical partitions	Parquet (Snappy by default) partitioned by date and hour	10
Control shuffles; broadcast small dimensions	Grid list broadcast to all workers; shuffle partitions set to 16	9 · P4
Window functions for rolling aggregates	Row-number de-duplication in Spark; 4-hour rolling sums and LAG windows in the checks	9, 12

Pillar 4 · Lakehouse and dimensional warehousing

Handbook requirement	How we meet it	Section
Delta Lake: ACID, schema enforcement, time travel, MERGE, partition pruning	Silver on Delta; MERGE on business keys restricted to the batch's dates; time travel shown in Snowflake	10
SCD Type 1 and Type 2	DIM_TRANSFORMERS (Type 1); DIM_METERS from the snapshot (Type 2)	11
Star schema with explicit fact grain	Two facts, two dimensions, alert fact; grain stated	11
Surrogate dimension keys	Partly: version IDs in the history table; numeric surrogate keys listed as a next step	11, 19
Additive, semi-additive and non-additive metrics	Classified and applied in the KPIs	11
Expose tables through a SQL / cloud OLAP engine	Snowflake	6, 9
Quality gates	Spark validation before writing Silver; 15 dbt tests plus a failing-on-purpose gate demo	14

Pillar 5 · Orchestration and analytics

Handbook requirement	How we meet it	Section
Airflow DAGs automating dependencies	smart_grid_pipeline: 5 dependent tasks every 15 minutes	13
Interactive dashboard with domain KPIs	Power BI, 4 pages	15
Retain workflow logs, results and quality evidence	Airflow logs, Silver batch logs, DLQ, dbt results, screenshots in this document	13, 14
Distinguish measured performance from assumed scale	Measured numbers, wall-clock vs event time	16

Project 04 · Smart energy acceptance criteria

Handbook requirement	How we meet it	Section
100,000+ meters in configuration and generated IDs	100,000 meters, IDs M-000001 to M-100000	8
15-minute timestamps with active power, reactive power and voltage	ts on :00/:15/:30/:45 with kw, kvar, voltage, kwh	8
All three anomaly families appear in the demonstration	Overload, line drop and theft detected and shown	12, 15
Relate meters to the grid hierarchy	Meter → transformer → feeder → substation → district	2, 8, 11
Transformer peak loading, anomaly flags, daily grid demand	KPI_TRANSFORMER_PEAK, ANOMALY_FLAGS, KPI_DAILY_DEMAND on the dashboard	15
Show the underlying simulated readings	Alert log by time and transformer; facts queryable in Snowflake	15
Raw-to-dashboard automated, validated, reproducible	Airflow + tests + fixed-seed config	13, 14, 20
Wall-clock vs event time; power vs energy; flags are not proof	Stated explicitly	11, 12, 16